Inteligencia Artificial y Neurociencias

Licenciatura en Tecnología Digital, UTDT, 2S-2026

Profesores: Agustín Gravano, Andrés Rieznik
Auxiliares: Gerardo Del Toro, Antonella Giordano Furchi

Fundamentos de LLMs #2

Bibliografía: Jurafsky & Martin, SLP, capítulo 9, "Masked Language Models",
y sección 13.3, "Details of the Encoder-Decoder Model"

Arquitecturas de Transformers

Decoder (ej: GPT, Claude, Llama)

A partir de una secuencia de tokens (un prompt), genera un token de salida por vez, de manera iterativa y autorregresiva. Son modelos generativos y causales: la información fluye de izquierda a derecha.

Encoder (ej: BERT, RoBERTa, RoBERTuito)

Recibe como entrada una secuencia de tokens y los mapea a una secuencia de embeddings contextuales de la misma longitud, que contienen información de toda la entrada. No genera texto: se usa para clasificar o tomar decisiones sobre la entrada (p. ej., análisis de sentimiento, NER).

Encoder-decoder (ej: BART, Whisper)

Un encoder procesa toda la secuencia de entrada; un decoder genera la secuencia de salida un token por vez, condicionada por el encoder mediante cross-attention.

Arquitecturas de Transformers

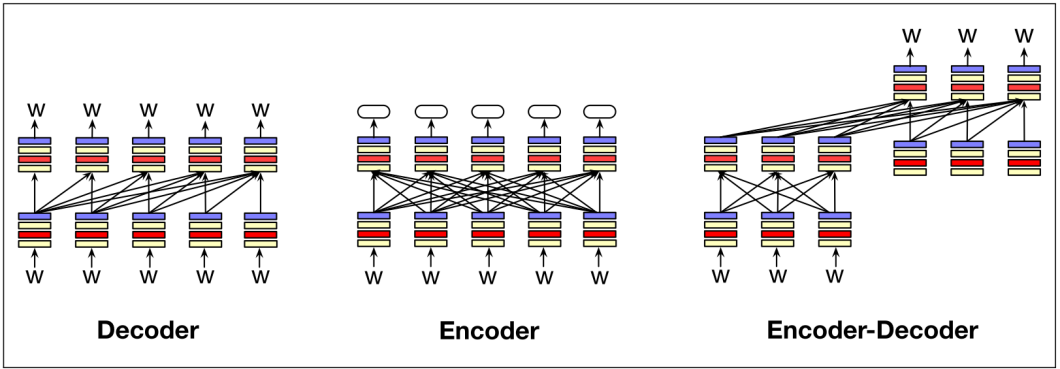


Figure 9.1 Three architectures for language models: decoders, encoders, and encoder-decoders. The arrows sketch out the information flow in the three architectures. Decoders take tokens as input and generate tokens as output. Encoders take tokens as input and produce an encoding (a vector representation of each token) as output. Encoder-decoders take tokens as input and generate a series of tokens as output.

Encoders bidireccionales (BERT)

- Usan self-attention para mapear secuencias de input embeddings $(x_1, \ldots, x_n)$ a secuencias de output embeddings de la misma longitud $(h_1, \ldots, h_n)$.
- Los output embeddings contienen información contextual de **toda** la secuencia de entrada (no solo de los tokens anteriores, como en GPT).
- Útiles para muchas tareas que requieren hacer clasificaciones o tomar decisiones basadas en los tokens y sus contextos.

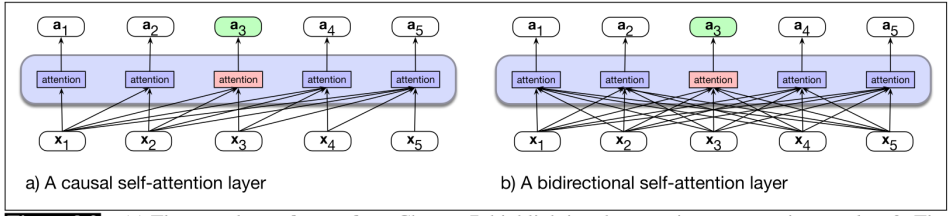


Figure 9.2 (a) The causal transformer from Chapter 7, highlighting the attention computation at token 3. The attention value at each token is computed using only information seen earlier in the context. (b) Information flow in a bidirectional attention model. In processing each token, the model attends to all inputs, both before and after the current one. So attention for token 3 can draw on information from following tokens.

Implementación: se elimina la máscara

La implementación es muy simple: se elimina la máscara que impide a la atención mirar a tokens futuros.

Decoder (causal):

$$\text{head} = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V$$

Encoder (bidireccional):

$$\text{head} = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} \cancel{+M}\right)V$$

0	$-\infty$	$-\infty$	$-\infty$	$-\infty$
0	0	$-\infty$	$-\infty$	$-\infty$
0	0	0	$-\infty$	$-\infty$
0	0	0	0	$-\infty$
0	0	0	0	0

Decoder

0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0

Encoder

Entrenamiento de BERT: Denoising + NSP

Denoising: se corrompe la entrada (algunos tokens se enmascaran; otros se reemplazan por otros al azar) y el modelo debe reconstruir los tokens originales.

NSP (Next Sentence Prediction): dadas dos oraciones, el modelo debe predecir si la segunda sigue a la primera en el texto original.

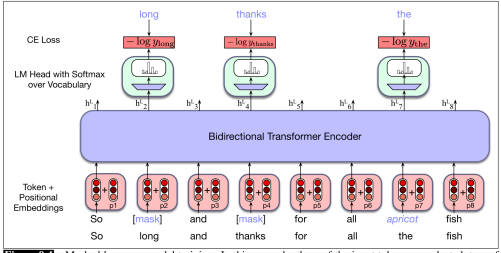


Figure 9.4 Masked language model training. In this example, three of the input tokens are selected, two of which are masked and the third is replaced with an unrelated word. The probabilities assigned by the model to these three items are used as the training loss. The other 5 tokens don't play a role in training loss.

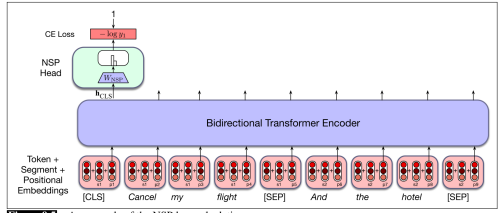


Figure 9.5 An example of the NSP loss calculation.

Ejemplo de entrenamiento: `[CLS] segmento A [SEP] segmento B [SEP]`

BERT se preentrena con dos objetivos simultáneos: MLM (predecir tokens enmascarados) y NSP (predecir si `B` sigue a `A`, desde $h_{\texttt{[CLS]}}$). Pérdida: $\mathcal{L} = \mathcal{L}_{\text{MLM}} + \mathcal{L}_{\text{NSP}}$

Fine-tuning, ejemplo 1: clasificación de sentimiento

Tarea: oración $\rightarrow$ {positivo, negativo, neutro}

Head: single feedforward layer con $W_c \in \mathbb{R}^{d \times 3}$

$$y = \text{softmax}\left(h^L_{\text{CLS}}\, W_c\right)$$

Entrenamiento:

- Datos etiquetados; cross-entropy loss entre el output y la respuesta correcta
- La matriz W_c se ajusta junto con el resto de la red
- Alternativa más barata: ajustar solo W_c, dejando el resto congelado
- Alternativa mejor: LoRA (lo veremos pronto)

Fine-tuning, ejemplo 1: clasificación de sentimiento

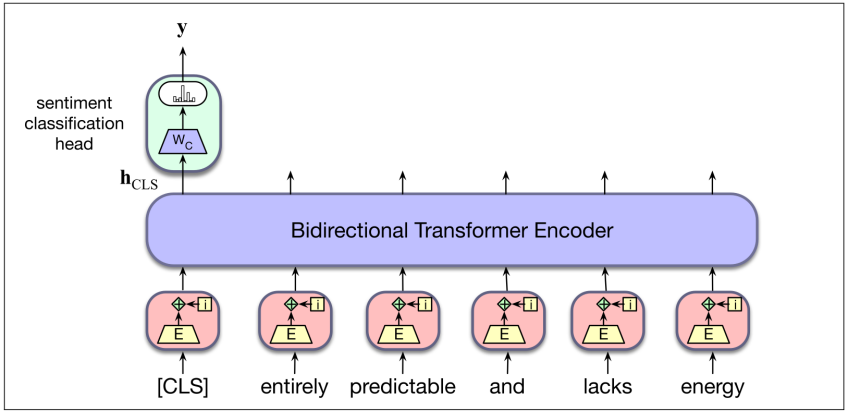


Figure 9.6 Sequence classification with a bidirectional transformer encoder. The output vector for the [CLS] token serves as input to a simple classifier.

Fine-tuning, ejemplo 2: NER

Tarea: oración $\rightarrow$ secuencia BIO

Gabriel	García	Márquez	nació	en	Aracataca	,	Colombia	,
B-PER	I-PER	I-PER	O	O	B-LOC	O	B-LOC	O

y	vivió	en	la	Ciudad	de	México	.
O	O	O	O	B-LOC	I-LOC	I-LOC	O

B: comienzo de una entidad **I**: continuación **O**: fuera de toda entidad
PER: persona **LOC**: lugar

Fine-tuning, ejemplo 2: NER

Head: single feedforward layer con $W_{\text{NER}} \in \mathbb{R}^{d \times k}$, con k = número de etiquetas para la tarea.

$$y_i = \text{softmax}\left(h_i^L \, W_{\text{NER}}\right) \;\rightarrow\; \text{distribución de probabilidades sobre las } k \text{ etiquetas}$$

$$t_i = \underset{k}{\text{argmax}} \; y_i[k]$$

Se ajusta la matriz W_{NER} junto con el resto de la red (o bien sola).

Observación: Claramente podría pensarse algo más sofisticado que esto para elegir la secuencia más probable (p. ej., Viterbi / CRF).

Fine-tuning, ejemplo 2: NER

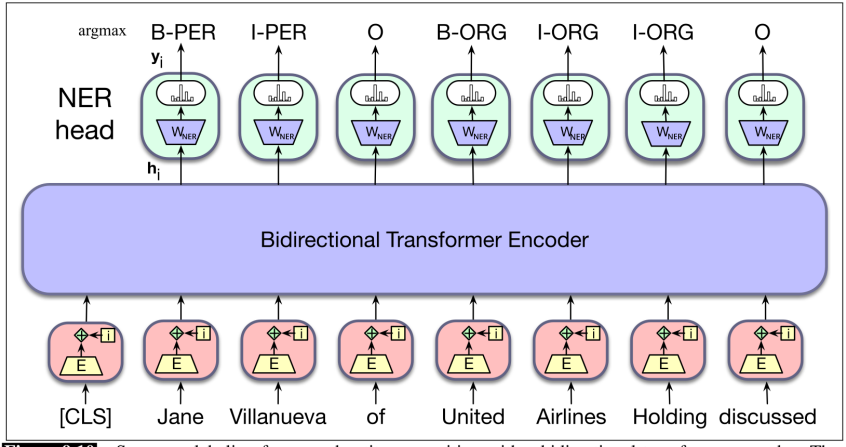


Figure 9.10 Sequence labeling for named entity recognition with a bidirectional transformer encoder. The output vector for each input token is passed to a simple k-way classifier.

Encoder-decoder: cross-attention

Cross-attention: Las queries vienen del decoder, pero las keys y values vienen del encoder.

$$Q = H^{dec}W^{Q}, \qquad K = H^{enc}W^{K}, \qquad V = H^{enc}W^{V}$$

$$\text{head} = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right)V$$

- Permite al decoder prestar atención a **todas** las palabras del source language mientras genera en el target language.
- La multi-head attention en el encoder no tiene masking, entonces presta atención adelante y atrás.

Encoder-decoder: traducción automática

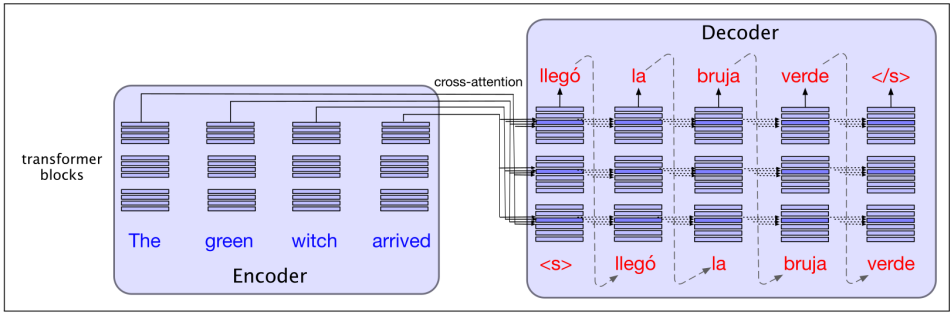


Figure 13.5 The encoder-decoder transformer architecture for machine translation. The encoder uses the transformer blocks we saw in Chapter 7, while the decoder uses a more powerful block with an extra **cross-attention** layer that can attend to all the encoder words. We'll see this in more detail in the next section.